0/1 Knapsack with Genetic Algorithms

Dr. Nirmala Shinde Baloorkar

Assistant Professor

Department of Computer Engineering, KJSSE

Learning Objectives

- Learn concept of Genetic algorithm
- Be able to apply the Genetic algorithm to given problem
- Understand merits and limitations of genetic algorithm

Why Genetic algorithms?[2]

- Disadvantages with Linear search
- **Discontinuous Search Spaces:**
  - While many algorithms require a smooth, continuous path to follow, GAs can navigate "jumpy" or disconnected landscapes where solutions are scattered.

Genetic algorithms

- A successor state is generated by combining two parent states
- **Step 1:** Start with *k* randomly generated states (population)
  - A state is represented as a string over a finite alphabet (often a string of 0s and 1s)
- **Step 2:** Evaluation function (fitness function). Higher values for better states.

Produce the next generation of states with:

- **Step 3:** selection
- **Step 4:** crossover, and
- **Step 5:** mutation

GA terminologies

- Crossover
- Mutation

Crossover

- Single point crossover : Choose any random breakup point and join the chromosomes

e.g.

X= x1 x2 x3 x4 x5 x6

Y= y1 y2 y3 y4 y5 y6

➔ Single point crossover (2:4)

X'= x1 x2 y3 y4 y5 y6

Y'= y1 y2 x3 x4 x5 x6

➔ Single point crossover (3:3)

X'= x1 x2 x3 y4 y5 y6

Y'= y1 y2 y3 x4 x5 x6

Crossover

- Two-point crossover : Choose any two random breakup points and join the chromosomes in interleaved manner

e.g.

X= x1 x2 x3 x4 x5 x6

Y= y1 y2 y3 y4 y5 y6

➔ Two-point crossover (2:4:2)

X'= x1 x2 y3 y4 x5 x6

Y'= y1 y2 x3 x4 y5 y6

➔ Two-point crossover (1:5:1)

X'= x1 y2 y3 y4 y5 x6

Y'= y1 x2 x3 x4 x5 x6

Mutation

- Bring a random change in parent chromosomes i.e. a small random tweak in the chromosome to get a new solution.
- Some mutation methods:
  - Flip a random bit
  - Exchange bits in a pair
  - Replace the genome with either lower or upper bound randomly

Genetic Algorithms- General Method

function GENETIC-ALGORITHM(Population, Fitness-FN)returns *an* individual

inputs: *population*, a set of individuals

FITNESS-FN, a function that measures the fitness of an individual

repeat

new-population ← empty set

loop for *i* **from 1 to** *Size(population)do*

x <- ***Random-Selection*(Population, Fitness-FN***)

x <- ***Random-Selection*(Population, Fitness-FN***)

child ←*REPRODUCE(x,y)*

if (small random probability) **then** *child* ← **Mutate***(child)*

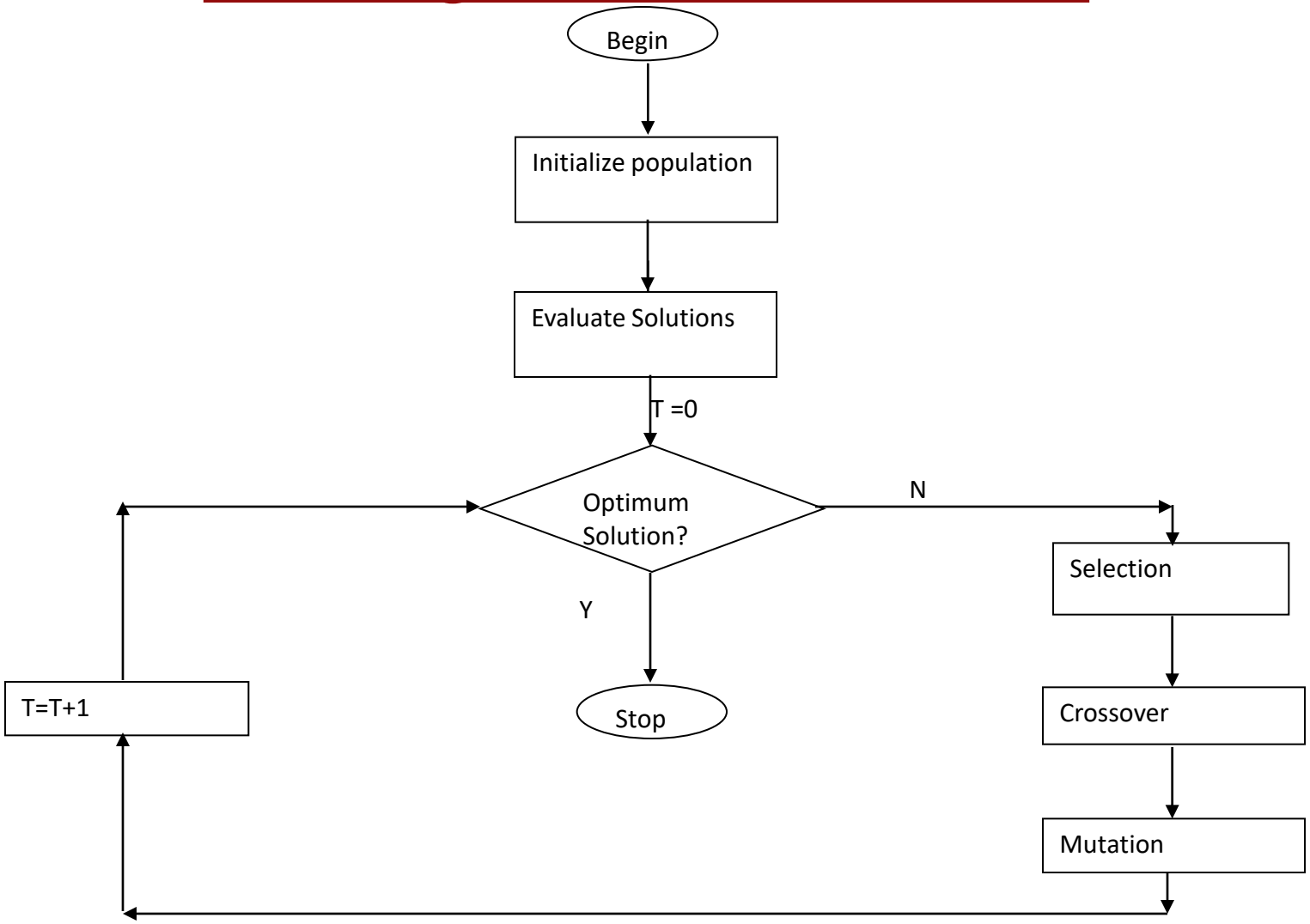
add *child* to *new-population*

population ← *new-population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to *FITNESS-FN*

Working Mechanism Of GAs



0/1 Knapsack

- **Problem definition:**

Given a knapsack with capacity M and N items with weights w_i and benefit b_i for 1<=i<=N, the problem is to

Maximize $\sum P_i * X_i$,

Subject to the condition

$\sum W_i * X^* <= M$ where Xi= 0 or 1.

0/1 Knapsack Solved with GA

Let N=4, (w,b)={ (7,42), (3,12), (4,40),(5,25)}, M=10

Step 1: Generate Random Initial Population with some Fitness Function (Profit)

	A	B	C	D	Weight	Profit
P	0	0	1	0	4	40
Q	0	1	0	1	8	37
R	1	1	0	0	10	54
S	1	1	1	0	14	00
T	0	1	1	0	7	52
U	0	0	0	1	5	25
V	0	0	1	0	3	12

Step 2: Fitness criteria

- **Total Fitness** is the sum of the fitness values of all individuals in the current population.
- In knapsack, weight is a constraint and profit is fitness
- If the constraint is violated (Total Weight > 10), we apply a **penalty**.
- The simplest penalty is setting the fitness to **0**, effectively "killing" that solution so it doesn't reproduce.
- Roulette fitness function.

Roulette Wheel Selection method

- Also known as Fitness Proportionate Selection
- The probability Pi of an individual i being selected is its fitness divided by the total fitness of the population:

$$P_i = \frac{F_i}{\sum_{j=1}^{N} F_j}$$

- Treat the fitness of each individual as a slice of a circular wheel.
- The larger the fitness, the larger the slice,
- And the higher the probability that the individual will be selected for reproduction.

Fitness value computation

	A	B	C	D	Weight	Profit
P	0	0	1	0	4	40
Q	0	1	0	1	8	37
R	1	1	0	0	10	54
S	1	1	1	0	14	00
T	0	1	1	0	7	52
U	0	0	0	1	5	25
V	0	0	1	0	3	12

- Total Fitness = 40+37+54+0+52+25+12 = 220
- P = 40/220 = 0.182
- Q = 37/220 = 0.168
- R = 54/220 = 0.245
- S = 0/220 = 0
- T = 52/220 = 0.236
- U = 25/220 = 0.114
- V = 12/220 = 0.055

Let's define fitness value >= 0.16

Step 2 continued

	A	B	C	D	Weight	Profit	Fitness value	Passed fitness?
P	0	0	1	0	4	40	0.182	Yes
Q	0	1	0	1	8	37	0.168	Yes
R	1	1	0	0	10	54	0.245	Yes
S	1	1	1	0	14	00	0	No
T	0	1	1	0	7	52	0.236	Yes
U	0	0	0	1	5	25	0.114	No
V	0	0	1	0	3	12	0.055	No

Step 3: Selection - Sort chromosomes as per fitness (Survival for the fittest)

	A	B	C	D	Weight	Profit	Fitness value
R	1	1	0	0	10	54	0.245
T	0	1	1	0	7	52	0.236
P	0	0	1	0	4	40	0.182
Q	0	1	0	1	8	37	0.168

Step 4: Reproduce with some crossover point

Before Crossover						
	A	B	C	D	Weight	Profit
R	1	1	0	0	10	54
T	0	1	1	0	7	52
P	0	0	1	0	4	40
Q	0	1	0	1	8	37

After (2:2) crossover						
	A	B	C	D	Weight	Profit
R1	1	1	1	0	14	??
T1	0	1	0	0	3	12
P1	0	0	0	1	5	25
Q1	0	1	1	0	7	52

(w,b)={ (7,42), (3,12), (4,40),(5,25)},

Discard the constraint violating chromosome?

- In a standard Genetic Algorithm, the answer is **no, you don't necessarily discard it immediately**,
- but you must handle it before you finalize the generation.

Step 5:-Mutation

- Flip Random bit

Before Mutation						
	A	B	C	D	Weight	Profit
R1	1	1	1	0	14	0
T1	0	1	0	0	3	12
P1	0	0	0	1	5	25
Q1	0	1	1	0	7	52
After Mutation						
	A	B	C	D	Weight	Profit
R1	1	1	0	0	10	0
T1	0	1	0	1	8	37
P1	1	0	0	1	12	67
Q1	0	1	1	0	7	52

- After first Generation, profit=67 whereas at the start profit was 54

Example 2

- Use a Genetic Algorithm to maximize

$$f(x)=x^2$$

where $x \in \{0,1,2,\ldots,31\}$.

Start with the initial population $x=\{12,25,5,19\}$.

Show one iteration of the GA (selection, crossover, mutation, and evaluation).

Example 2 - Step 1: Initial Population

- Each chromosome is a **5-bit binary string**.

	Chromosome	Decimal Value (x)
P	01100	12
Q	11001	25
R	00101	5
s	10011	19

Example 2 - Fitness Function Evaluation

	Chromosome	Decimal Value (x)	$f(x)=x^2$
P	01100	12	144
Q	11001	25	625
R	00101	5	25
s	10011	19	361

- Total Fitness $\sum f(x)$

$=144+625+25+361=1155$

- Average Fitness $=1155/4$

$=288.75$

- Maximum fitness: 625

Example 2 - Selection (Roulette Wheel Selection)

	Chromosome	Decimal Value (x)	$f(x)=x^2$	Probability
P	01100	12	144	0.1247
Q	11001	25	625	0.5411
R	00101	5	25	0.0216
s	10011	19	361	0.3126

	Chromosome	Decimal Value (x)	$f(x)=x^2$	Probability	Excepted Count	Actual Count
P	01100	12	144	0.1247	0.4987	1
Q	11001	25	625	0.5411	2.1645	2
R	00101	5	25	0.0216	0.0866	0
s	10011	19	361	0.3126	1.2502	1

Example 2 - Crossover

	Mating Pool	Crossover Point	Offspring after crossover	X value	$f(x)=x^2$
P1	01100	4	01101	13	169
Q1	11001		11000	24	576
Q1	11001	2	11011	27	729
s1	10011		10001	17	289

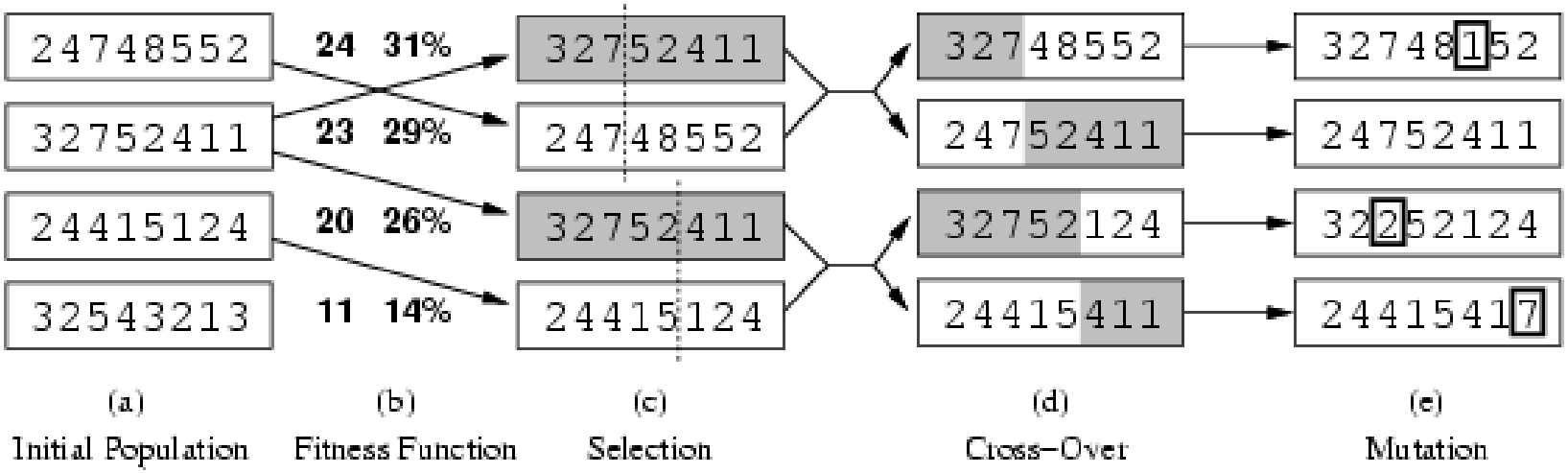
Example 2 - Mutation

- Flip Random Bit

	Offspring after the crossover	Mutation	New offspring	X value	$f(x)=x^2$
P1	01101	1st bit	11101	29	841
Q1	11000	-	11000	24	576
Q1	11011	-	11011	27	729
s1	10001	3rd and 5th bit	10100	20	400

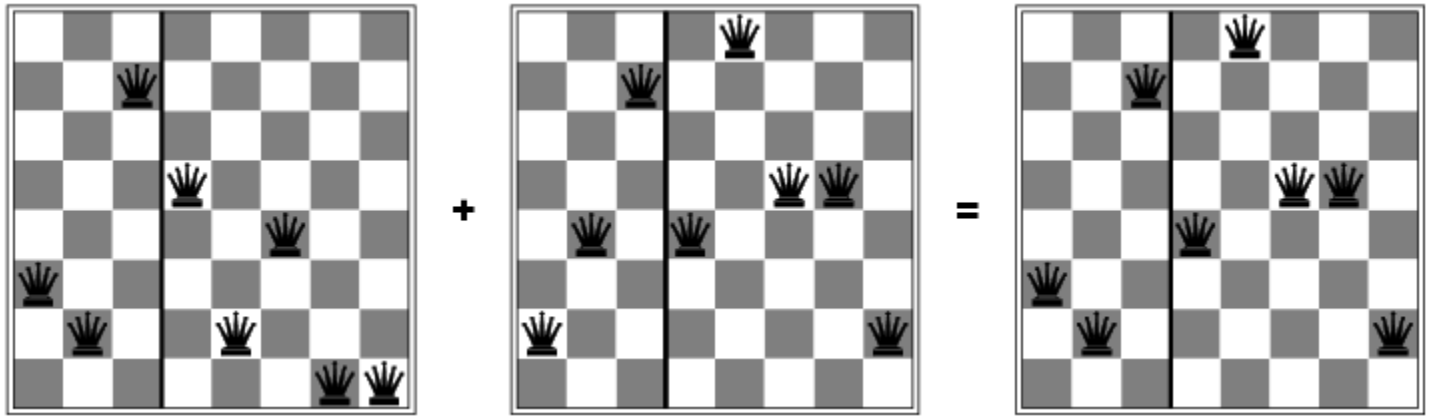
- After one iteration maximum is 841.

Genetic algorithms: N-Queens



- Fitness function: number of non-attacking pairs of queens (min = 0, max = 8 × 7/2 = 28)
- 24/(24+23+20+11) = 31%
- 23/(24+23+20+11) = 29%

Genetic algorithms: N-Queens



Exercise - 0/1 Knapsack Solved with GA

Let N=4, (w,b)={ (7,5), (2,4), (1,7),(9,2)}, M=15

Step 1: Generate Random Initial Population with some Fitness Function (Profit)

	A	B	C	D	Weight	Profit
P	0	1	0	0	2	4
Q	0	1	0	1	11	6
R	1	1	1	1	19	18
S	0	0	1	0	1	7
T	1	0	0	0	7	5
U	1	0	1	0	8	12
V	1	1	0	0	9	9

Exercise

- Use a Genetic Algorithm to maximize

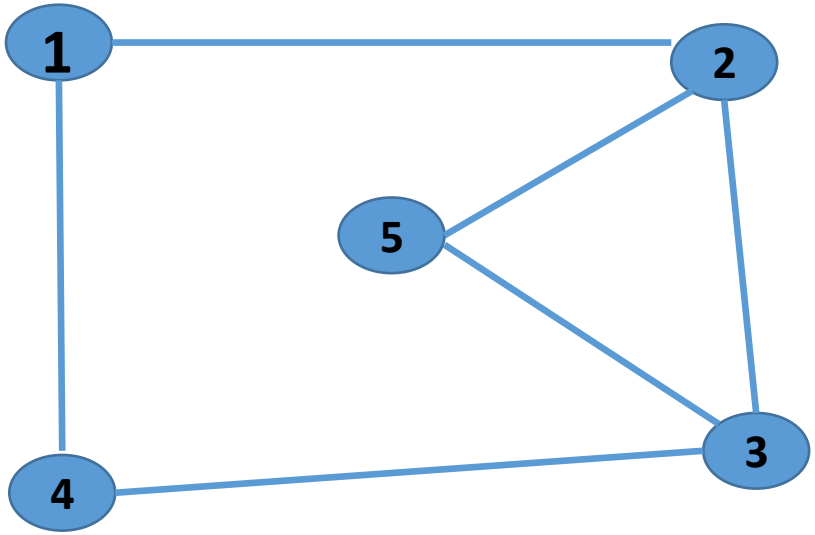
$$f(x)=x^2$$

where $x \in \{0,1,2,\ldots,31\}$.

Start with the initial population $x=\{13,24,8,16\}$.

Show one iteration of the GA (selection, crossover, mutation, and evaluation).

Solve- Graph coloring problem



Merits & Demerits

- Solution in linear amount of time.[4]
- Short running time
- Memory usage depends on number of individuals but is generally manageable
- Limitation - Doesn't necessarily find the global optimal solution
- Quality of solution depends on choosing an efficient representation + letting it run long enough
- Reasonably simple to implement, design decisions may be a little more complex, especially if you don't have significant experience with GAs[2]

References

1. Rich, Knight, “Artificial Intelligence”, Tata McGraw Hill Publication, 2nd Edition
2. http://stackoverflow.com/questions/14848874/which-is-the-best-method-between-genetic-algorithm-and-dynamic-programming-to-so, retrieved April 02,2015
3. Russel, Norving, “Artificial Intelligence: A modern Approach”, Pearson Education, 2nd Ed
4. Singh, R.P., "Solving 0–1 Knapsack problem using Genetic Algorithms," *Communication Software and Networks (ICCSN), 2011 IEEE 3rd International Conference on* , vol., no., pp.591,595, 27-29 May 2011, doi: 10.1109/ICCSN.2011.6013975